



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Facultat d'Informàtica de Barcelona



PLATAFORMA WEB INTEGRAL PER A LA CONFIGURACIÓ, MANTENIMENT I VISUALITZACIÓ DE SISTEMES DE VIDEOVIGILÀNCIA IP

JAN ROSELL CABA

Director/a

FERNANDO GALLEGO VILALTELLA (LANACCESS TELECOM, SA)

Ponent: CARLES FARRE TOST (Departament d'Enginyeria de Serveis i Sistemes d'Informació)

Titulació

Grau en Enginyeria Informàtica (Enginyeria del Software)

Memòria del treball de fi de grau

Facultat d'Informàtica de Barcelona (FIB)

Universitat Politècnica de Catalunya (UPC) - BarcelonaTech

Resum

Aquest projecte presenta el disseny i desenvolupament de **Web Core View**, una plataforma web d'última generació destinada a la gestió centralitzada de videogravadors de seguretat. El treball neix de la necessitat de Lanaccess Telecom, SA de migrar la seva infraestructura de gestió d'aplicacions natives cap a una solució basada en navegador que garanteixi ubiqüitat i sostenibilitat. La solució s'ha implementat com una *Single Page Application* (SPA) utilitzant **Angular 19** i el nou paradigma de reactivitat basat en **Signals**. Els dos pilars tecnològics més rellevants del projecte són el motor de configuració dinàmic (*Schema-driven*), que permet desacoblar la interfície de l'evolució del *firmware*, i la integració del protocol **WebRTC** per a la visualització de vídeo de baixa latència ($< 300\text{ms}$) sense necessitat de *plugins* externs. Els resultats validen que la solució web iguala o millora el rendiment de les eines clàssiques de l'empresa.

Abstract

This project presents the design and development of **Web Core View**, a state-of-the-art web platform for the centralized management of security video recorders. The project arises from the need of Lanaccess Telecom, SA to migrate its management infrastructure from native desktop applications to a browser-based solution that ensures ubiquity and sustainability. The solution has been implemented as a *Single Page Application* (SPA) using **Angular 19** and the new reactivity paradigm based on **Signals**. The two most significant technological pillars of the project are the dynamic configuration engine (*Schema-driven*), which allows decoupling the interface from firmware evolution, and the integration of the **WebRTC** protocol for low-latency video streaming ($< 300\text{ms}$) without the need for external plugins. The results validate that the web solution matches or improves the performance of the company's legacy tools.

Índex

1	Introducció i Context	1
1.1	Context del projecte	1
1.2	Identificació del problema i motivació	1
1.3	Estat de l'art i alternatives existents	2
1.3.1	Estat de l'art	2
1.3.2	Taula comparativa	2
1.4	Anàlisi d'alternatives i justificació de la solució	2
1.4.1	Alternativa de vídeo: WebRTC vs. HLS	3
1.4.2	Alternativa de configuració: Estàtica vs. Schema-driven	3
1.4.3	Alternativa de gestió d'estat: Redux (NgRx) vs. Angular Signals	3
1.5	Solució proposada i valor afegit	3
1.6	Objectius del projecte	4
1.6.1	Objectiu general	4
1.6.2	Objectius específics	4
1.7	Abast i Obstacles	4
1.7.1	Abast funcional	4
1.7.2	Obstacles detectats	5
1.8	Actors implicats (Stakeholders)	5
2	Gestió del projecte	6
2.1	Metodologia i eines de treball	6
2.2	Planificació temporal	7
2.2.1	Descripció de les fases i tasques	7
2.2.2	Recursos identificats	7
2.2.3	Estimació d'hores i Diagrama de Gantt	7
2.2.4	Gestió del risc i desviacions reals	8
2.2.5	Desviacions i impacte econòmic	8
2.3	Anàlisi econòmica	9
2.3.1	Costos de personal	9
2.3.2	Costos totals i control de gestió	9
2.4	Informe de sostenibilitat	9
3	Especificació de Requisites	10
3.1	Metodologia d'extracció de requisits	10
3.2	Identificació d'actors	10
3.3	Històries d'Usuari (User Stories)	11

3.3.1	Mòdul de Visualització	11
3.3.2	Mòdul de Configuració (Schema-Driven)	11
3.4	Requisits Funcionals (RF)	11
3.4.1	RF-1: Gestió de l'Accés i Seguretat	12
3.4.2	RF-2: Visualització Multimèdia	12
3.4.3	RF-3: Motor de Configuració Dinàmica	12
3.5	Requisits No Funcionals (RNF)	12
3.5.1	RNF-1: Rendiment i Eficiència	13
3.5.2	RNF-2: Robustesa i Disponibilitat	13
3.5.3	RNF-3: Compatibilitat i Portabilitat	13
3.6	Casos d'Ús detalla'ts	13
3.6.1	UC-01: Gestió de l'actualització de firmware	13
3.6.2	UC-02: Consulta de gravacions mitjançant Timeline	13
3.7	Restriccions del sistema	13
3.8	Marc normatiu i regulacions	16
3.8.1	Protecció de Dades (RGPD)	16
3.8.2	Seguretat en la comunicació	16
3.8.3	Normativa de seguretat privada	16
4	Arquitectura i Disseny	18
4.1	Arquitectura Global del Sistema	18
4.1.1	Capa de comunicació i Microserveis	18
4.2	Disseny del Frontend (Angular 19)	19
4.2.1	Arquitectura Modular i Standalone Components	19
4.2.2	Gestió de l'Estat amb Angular Signals	19
4.3	El Motor Schema-Driven	19
4.3.1	Lògica de funcionament	19
4.4	Disseny de la Persistència i dades	20
5	Implementació	21
5.1	Entorn de desenvolupament i tecnologies	21
5.2	Mòdul d'Autenticació i Core	21
5.2.1	Protecció de rutes i Resolvers	21
5.3	Implementació del Motor Schema-Driven	22
5.3.1	El component genèric de configuració	22
5.4	Visualització de Vídeo en Temps Real (WebRTC)	22
5.4.1	Negociació SDP i intercanvi de candidats	22
5.5	Sistema d'Alarmes i Comunicació asíncrona	22
5.6	Mòdul de Manteniment i Logs	23
6	Validació i Proves	24
6.1	Estratègia de proves	24
6.2	Proves Unitàries i de Lògica	24
6.2.1	Validació del Parser de metadades	24
6.3	Resultats de rendiment i latència	25
6.4	Proves de Robustesa i Comunicació asíncrona	25
6.4.1	Escenari de tall de connectivitat	25

6.5	Validació d'Acceptació (UAT)	26
7	Integració de Coneixements	27
7.1	Enginyeria del Software (ES)	27
7.2	Xarxes (X)	27
7.3	Sistemes Operatius (SO)	27
7.4	Projecte de Programació (PROP)	28
7.5	Aprenentatge autònom	28
8	Conclusions i Treball Futur	29
8.1	Assoliment dels objectius	29
8.2	Treball futur	29
8.3	Conclusions personals	30
A	Annexos	31

Índex de taules

1.1	Comparativa de solucions de mercat.	2
2.1	Estimació d'hores i dependències de les tasques.	8
2.2	Costos de recursos humans per rol.	9
3.1	Especificació detallada del Cas d'Ús UC-01.	15
3.2	Especificació detallada del Cas d'Ús UC-02.	15
6.1	Comparativa de latència segons protocol de comunicació.	25

Índex de figures

2.1	Diagrama de Gantt del projecte.	8
3.1	Diagrama UML de Casos d'Ús.	14

Capítol 1

Introducció i Context

En aquest primer capítol es defineix el marc en el qual es desenvolupa el projecte, s'analitza la problemàtica actual de l'empresa Lanaccess i es justifica la necessitat d'una nova plataforma web. Així mateix, es detallen els objectius, l'abast i els actors implicats en el cicle de vida del software.

1.1 Context del projecte

El present Treball de Final de Grau (TFG) consisteix en el disseny i desenvolupament de **Web Core View**, una aplicació web d'última generació destinada a la gestió centralitzada dels videogravadors de l'empresa Lanaccess Telecom, SA. El projecte es realitza en col·laboració amb aquesta empresa tecnològica, especialitzada en el desenvolupament de solucions professionals de videovigilància i seguretat electrònica.

El treball s'emmarca dins de la **Modalitat B** del TFG i dins de l'especialitat d'Enginyeria de Software, posant èmfasi en arquitectures frontend modernes, seguretat en la comunicació i integració de protocols de *streaming* en temps real. L'objectiu principal és liderar la transició digital de l'empresa, substituint el programari de gestió basat en aplicacions natives per a Windows per una solució *Single Page Application* (SPA) basada en l'ecosistema Angular.

1.2 Identificació del problema i motivació

Històricament, la gestió operativa i tècnica dels gravadors de vídeo s'ha realitzat mitjançant aplicacions d'escriptori pesades. Aquest model presenta avui dia una obsolescència tecnològica que es tradueix en els següents problemes:

- **Costos de manteniment elevats:** Les aplicacions natives obliguen a realitzar desplegaments manuals en cada terminal de client per a cada actualització, dificultant l'escalabilitat en grans infraestructures.
- **Rigidesa davant hardware heterogeni:** Els gravadors moderns posseeixen capacitats i *firmwares* molt diversos. Les interfícies estàtiques actuals no poden adaptar-se eficientment a la disparitat de paràmetres (IP, ports, lògiques d'alarma) de cada equip.

- **Obsolescència d'experiència d'usuari (UX):** Les eines actuals requereixen una corba d'aprenentatge elevada i no compleixen els estàndards de reactivitat i claredat que exigeixen els operadors de seguretat.

1.3 Estat de l'art i alternatives existents

1.3.1 Estat de l'art

Actualment, el mercat de la videovigilància IP està dominat per grans fabricants com Hikvision o Dahua, que ofereixen interfícies web integrades. El principal inconvenient d'aquestes solucions és que sovint encara depenen de protocols propietaris o de la instal·lació de *plugins* externs (com els antics ActiveX o extensions específiques) per poder visualitzar el vídeo, cosa que limita la seva compatibilitat a navegadors o sistemes operatius concrets.

Aquesta dependència està desapareixent amb la tendència actual cap al **Plugin-less video**. L'aparició d'estàndards com **WebRTC** ha suposat un punt d'inflexió, permetent la transmissió de fluxos multimèdia d'alta fidelitat i baixa latència de forma nativa en el navegador, eliminant les barreres de seguretat i instal·lació que imposaven les tecnologies anteriors.

D'altra banda, els líders de programari VMS (*Video Management System*) com Milestone o Genetec ofereixen solucions molt potents però amb arquitectures extremadament pesades i costos de llicenciament elevats, enfocades sovint a clients amb gran capacitat de còmput local.

1.3.2 Taula comparativa

A la Taula 1.1 es mostra una comparació entre la solució nativa anterior, els líders de mercat i la proposta de Web Core View.

Taula 1.1: Comparativa de solucions de mercat.

Característica	App Windows	Market Leaders	Web Core View
Accessibilitat	Baixa (Windows)	Mitjana (<i>Plugins</i>)	Alta (Navegador)
Manteniment	Difícil (Manual)	Complex (Servidors)	Fàcil (Centralitzat)
Interfície (UX)	Obsoleta	Professional/Complexa	Moderna i Intuïtiva
Configuració	Estàtica	Propietària	Dinàmica (<i>Schema-driven</i>)

Font: Elaboració pròpia.

1.4 Anàlisi d'alternatives i justificació de la solució

Per garantir l'èxit i la sostenibilitat del projecte, s'han avaluat diferents camins tecnològics per als tres pilars fonamentals del sistema:

1.4.1 Alternativa de vídeo: WebRTC vs. HLS

- **HLS (HTTP Live Streaming):** Ideal per a retransmissions massives on la latència no és crítica (com Netflix o YouTube). En videovigilància, un retard de 2 a 5 segons és inacceptable per a l'operació tècnica.
- **WebRTC (Web Real-Time Communication):** Permet una comunicació de baixa latència (<300ms) mitjançant UDP.
- **Justificació:** S'ha escollit **WebRTC** com a protocol principal per a la visualització en directe per igualar l'experiència de les aplicacions natives, mantenint HLS només com a mecanisme de *fallback* i per a la reproducció de gravacions.

1.4.2 Alternativa de configuració: Estàtica vs. Schema-driven

- **Formularis estàtics:** Programar una pantalla per a cada menú de configuració és més ràpid a curt termini però genera un deute tècnic insostenible, ja que cada canvi en el *firmware* obligaria a actualitzar el *frontend*.
- **Motor Schema-driven:** El *frontend* actua com un intèrpret que renderitza la interfície basant-se en un esquema JSON rebut del servidor.
- **Justificació:** S'ha optat per la solució **Schema-driven** per garantir la compatibilitat futura amb nous models de gravador de Lanaccess sense necessitat de modificar el codi de l'aplicació web.

1.4.3 Alternativa de gestió d'estat: Redux (NgRx) vs. Angular Signals

- **Redux/NgRx:** Solucions robustes per a aplicacions gegantines, però amb un *boilerplate* elevat i un consum de recursos que pot penalitzar dispositius amb hardware limitat.
- **Angular Signals:** La nova eina nativa d'Angular 19 per a la gestió de la reactivitat granular.
- **Justificació:** S'ha seleccionat **Signals** per la seva lleugeresa i eficiència en el renderitzat. En un entorn *embedded* on la CPU s'ha de prioritzar per al vídeo, la reactivitat nativa de Signals és l'opció més òptima.

1.5 Solució proposada i valor afegit

Web Core View es proposa com una solució integral que aprofita el màxim potencial d'**Angular 19**. L'elecció d'aquesta versió, altament recent, respon a la voluntat d'utilitzar tecnologies que defineixen el futur del desenvolupament web. El seu valor diferencial resideix en:

- **Arquitectura Schema-driven:** La interfície interpreta un esquema JSON enviat pel gravador i genera automàticament els controls de configuració, assegurant compatibilitat amb qualsevol versió de hardware.
- **Gestió d'estat amb Signals:** L'ús d'**Angular Signals** representa un canvi de paradigma en la reactivitat. Aquesta funcionalitat permet una gestió granular dels canvis, optimitzant el rendiment del *frontend* i garantint una fluïdesa absoluta fins i tot amb múltiples fluxos de vídeo simultanis, evitant sobrecàrregues de CPU innecessàries al terminal del client.
- **Control de concurrència:** Implementació d'un servei de bloqueig en temps real per evitar conflictes d'edició entre administradors.
- **Arquitectura d'esdeveniments:** Ús de *WebSockets* (WSS) per rebre notifikacions d'alarmes de forma instantània.

1.6 Objectius del projecte

1.6.1 Objectiu general

Dissenyar i desenvolupar una plataforma web SPA modular, reactiva i d'alt rendiment que centralitzi la gestió tècnica i operativa dels dispositius Lanaccess, eliminant la dependència de programari natiu.

1.6.2 Objectius específics

- Desenvolupar un motor de configuració dinàmic basat en esquemes metadada.
- Integrar un mòdul de visualització híbrid compatible amb WebRTC i HLS (H264/H265).
- Implementar un sistema de gestió d'alarmes en temps real mitjançant comunicació bidireccional.
- Facilitar el manteniment remot (firmware, llicències i logs) a través del navegador.

1.7 Abast i Obstacles

1.7.1 Abast funcional

L'aplicació ha d'oferir un cicle complet de gestió: des de l'autenticació segura de l'usuari fins a la monitorització de càmeres en directe, la cerca de gravacions històriques i la configuració avançada de xarxa i paràmetres de sistema.

1.7.2 Obstacles detectats

S'han identificat reptes tècnics que afecten diferents actors:

- **Complexitat del vídeo natiu:** El repte de visualitzar H.265 sense *plugins*. Afecta al **Projectista** (recerca de llibreries) i a l'**Usuari final** (potencial latència).
- **Sincronització en temps real:** Requereix una gestió d'estat robusta perquè l'**Operador** vegi alarmes sense retard.
- **Seguretat i NGINX:** Configuració de CORS i certificats que afecta al desplegament de l'**Empresa**.

1.8 Actors implicats (Stakeholders)

1. **Usuari final (Operador):** Monitoritza càmeres i gestiona incidents.
2. **Administrador tècnic:** Configura el sistema i realitza manteniment.
3. **Lanaccess Telecom:** Proporciona el *hardware* i coneixement de negoci.
4. **Projectista (Estudiant):** Responsable únic de l'anàlisi, disseny i implementació.
5. **Equip docent (Director i Ponent):** Supervisen el rigor acadèmic i tècnic.

Capítol 2

Gestió del projecte

En aquest capítol es descriu el marc metodològic utilitzat per al desenvolupament del projecte, la planificació temporal detallada de les tasques i l'anàlisi econòmica i de sostenibilitat del sistema.

2.1 Metodologia i eines de treball

El projecte es desenvolupa en un entorn professional dins de l'equip d'*Embedded* de Lanaccess. S'ha optat per una **metodologia àgil basada en Kanban**, utilitzant l'eina *Businessmap* per a la gestió visual del flux de treball. Aquesta elecció permet una gran flexibilitat i una resposta ràpida davant de canvis en els requeriments tècnics.

Els pilars del seguiment del projecte són:

- **Reunions de sincronització (Daily):** Sessions diàries per coordinar el progrés, identificar bloquejos i alinear el *frontend* amb l'estat del *firmware*.
- **Seguiment acadèmic i professional:** El director del TFG supervisa l'evolució tècnica diàriament, mentre que es manté una comunicació constant amb el tutor de la universitat per garantir el rigor acadèmic.
- **Gestió de tasques:** Control del cicle de vida de cada funcionalitat des de la columna *To Do* fins a *Done*.

Respecte a l'enginyeria de software, se segueixen els estàndards següents:

- **Control de versions:** Ús de **Git** amb el model **GitFlow** (branques *master*, *develop* i *feature*). El repositori es troba allotjat en un GitLab local.
- **Stack tecnològic:** Angular 19 (Signals i Standalone Components) per al *frontend* i implementacions en C++ 22 per a la comunicació de baix nivell en el *backend*.
- **Qualitat:** Aplicació de principis SOLID i arquitectura modular.

2.2 Planificació temporal

2.2.1 Descripció de les fases i tasques

El projecte s'estructura en sis fases lògiques pactades amb l'empresa:

1. **Gestió i Formació (GF):** Inclou l'autoformació en Angular 19 i la gestió administrativa de l'assignatura GEP.
2. **Mòdul d'Autenticació i Core (AC):** Desenvolupament del *login*, la *home page* i el sistema base de *logs* i alertes.
3. **Manteniment i Configuració Base (MC):** Gestió de llicències, actualització de *firmware* i integració del motor de base de dades.
4. **Alarmes i Sistema de Vídeo Directe (AV):** Implementació del motor d'alarmes via WSS i visualització en temps real.
5. **Mòdul de Gravacions i Configuració Avançada (GV):** Reproducció d'històrics (HLS) i motor de configuració dinàmica *Schema-driven*.
6. **Finalització i Memòria (FM):** Fase de correcció d'errors, repàs de qualitat i redacció final.

2.2.2 Recursos identificats

Per a l'execució del projecte es requereixen els següents recursos:

- **Humans (Rols):** Cap de Projecte (planificació), Analista (disseny d'arquitectura), Programador (codificació) i Tester (validació).
- **Materials:** Ordinador portàtil (ASUS Expertbook), videogravador Lanaccess de proves i espai de despatx equipat.
- **Software:** VS Code, Git, Angular 19, RxJS i llibreries de vídeo (WebRTC, HLS.js).

2.2.3 Estimació d'hores i Diagrama de Gantt

S'ha estimat una càrrega total de **540 hores** (18 ECTS). La Taula 2.1 resumeix el repartiment temporal segons el planning oficial.

Taula 2.1: Estimació d'hores i dependències de les tasques.

ID	Tasca	Mes/Setmana	Dependència	Hores
GF1	Formació Angular 19	Desembre	-	40
AC1	Login	Gener S1	GF1	20
AC2	Home Page	Gener S2	AC1	20
AV1	Core Alarm (WSS)	Feb S4 - Mar S1	AC3	50
AV3	Càmeres Directe	Març S3 - S4	AC2	50
GV2	System Config Avançat	Abr S3 - Maig S1	MC2	60
FM2	Memòria TFG	Transversal	-	80
TOTAL				540

Font: Elaboració pròpia.

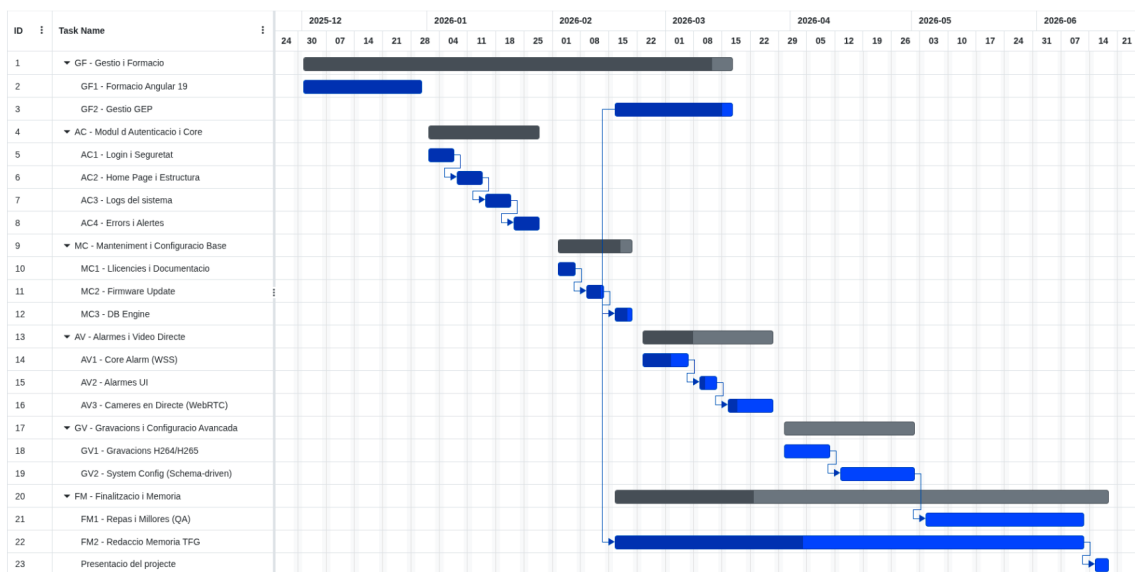


Figura 2.1: Diagrama de Gantt del projecte.

Font: Elaboració pròpia.

2.2.4 Gestió del risc i desviacions reals

S'han previst plans de contingència per a retards en la formació o incompatibilitats en els protocols de vídeo (WebRTC). Actualment, s'han detectat desviacions significatives en el mòdul de reproducció de gravacions, ja que la integració del *backend* ha presentat una complexitat superior a la prevista, especialment pel que fa a la precisió requerida pel còdec H.265. Així mateix, el disseny del motor de configuració dinàmica (definició d'esquemes i mapeig de camps) ha requerit una fase d'arquitectura més extensa de l'estimada inicialment.

2.2.5 Desviacions i impacte econòmic

Aquestes desviacions han tingut un impacte directe en el consum de recursos humans, especialment en els rols d'**Analista** i **Programador**. El temps addicional dedicat a

la recerca i implementació del suport H.265 sense *plugins* ha suposat un increment aproximat de **45 hores** sobre la planificació inicial.

A nivell econòmic, aquest sobrecost teòric s'ha gestionat mitjançant el **marge de contingència del 10%** definit en el pressupost inicial. Tot i que el cost real del personal ha augmentat, el pressupost final no s'ha vist compromès gràcies a aquesta reserva, evitant així la necessitat de finançament addicional o de retallar funcionalitats crítiques de l'abast del projecte.

2.3 Anàlisi econòmica

L'estimació econòmica s'ha realitzat assumint rols professionals i preus de mercat actuals, afegint un factor d'1,35 per cobrir la Seguretat Social.

2.3.1 Costos de personal

Taula 2.2: Costos de recursos humans per rol.

Rol	Sou Brut/h (€)	Cost Real/h (€)	Hores	Total (€)
Project Manager	25,00	33,75	90	3.037,50
Analista	20,00	27,00	100	2.700,00
Programador	18,00	24,30	250	6.075,00
Tester	13,00	17,55	100	1.755,00
TOTAL CPA			540	13.567,50

2.3.2 Costos totals i control de gestió

Sumant l'amortització del hardware (210,00 €), els costos indirectes d'oficina (1.150,00 €) i un marge de contingència del 10 %, el pressupost final estimat és de **16.810,40 €**.

El control de gestió es realitza mitjançant la fórmula de desviació de costos:

$$\text{Desviació} = (\text{Hores Planificades} - \text{Hores Reals}) \times \text{Cost/h}$$

2.4 Informe de sostenibilitat

- **Dimensió Econòmica:** La solució *Schema-driven* permet adaptar el software a nous models de gravadors sense cost addicional, allargant la vida útil del producte.
- **Dimensió Ambiental:** L'ús de la plataforma web redueix els desplaçaments físics de tècnics a les instal·lacions del client, minimitzant la petjada de carboni.
- **Dimensió Social:** Millora la interfície de treball dels operadors de seguretat, reduint l'estrès operatiu en situacions d'incident crític.

Capítol 3

Especificació de Requisits

L'especificació de requisits és la pedra angular del projecte *Web Core View*. En tractar-se d'un sistema de gestió per a dispositius de seguretat crítica, la definició de les funcionalitats no només s'ha centrat en l'experiència d'usuari (UX), sinó també en la robustesa de la comunicació asíncrona i el desacoblament entre el hardware i la interfície.

En aquest capítol es detallen els actors, les històries d'usuari mitjançant la metodologia *Product Backlog*, i l'especificació detallada dels requisits funcionals i no funcionals.

3.1 Metodologia d'extracció de requisits

Els requisits aquí exposats s'han obtingut mitjançant un procés iteratiu amb l'equip d'Embedded de Lanaccess. S'han realitzat sessions d'enginyeria inversa sobre les aplicacions natives de Windows existents i entrevistes amb tècnics de camp per identificar els "punts de dolor" de les eines actuals.

3.2 Identificació d'actors

S'han definit tres actors principals, classificant-los segons el seu nivell d'interacció i privilegis dins del sistema:

Administrador de Sistema: Actor amb privilegis totals sobre el videogravador. El seu objectiu és la posada en marxa de l'equip i el manteniment correctiu. Les seves responsabilitats inclouen:

- Configuració de paràmetres de xarxa (IP, DNS, Gateways).
- Gestió del cicle de vida del hardware (Reiniciis, actualització de firmware).
- Administració de comptes d'usuari i permisos.

Operador de Seguretat: Usuari final que utilitza la plataforma per a la monitorització. La seva interacció se centra en el consum de recursos multimèdia:

- Visualització de vídeo en temps real (*Live*).

- Navegació i exportació de gravacions històriques.
- Supervisió i silenciament d'alarmes en temps real.

Videogravorador (System Actor): Malgrat ser un element de hardware, en aquesta arquitectura actua com un usuari actiu que:

- Dicta la estructura de la interfície mitjançant esquemes JSON.
- Notifica esdeveniments de forma proactiva via WebSockets.
- Gestiona el control de concurrència per evitar col·lisions de dades.

3.3 Històries d'Usuari (User Stories)

S'utilitzen les *User Stories* per definir el valor que cada funcionalitat aporta a l'usuari final. Cada història inclou els seus criteris d'acceptació.

3.3.1 Mòdul de Visualització

US-01: Streaming d'alta fidelitat

Com a Operador, **vull** veure el vídeo de qualsevol càmera connectada, **per tal de** vigilar l'espai protegit en temps real.

- *Criteri 1:* La latència punta a punta (glass-to-glass) ha de ser inferior a 300ms.
- *Criteri 2:* El sistema ha de detectar automàticament si el navegador suporta WebRTC o ha de commutar a HLS.

US-02: Gestió de Mosaic

Com a Operador, **vull** visualitzar fins a 4 càmeres simultàniament, **per tal de** tenir una visió global de la instal·lació.

3.3.2 Mòdul de Configuració (Schema-Driven)

US-03: Configuració sense recàrrega de codi

Com a Administrador, **vull** que els nous paràmetres afegits al firmware apareguin a la web sense haver de reprogramar el frontend.

- *Criteri 1:* El frontend ha d'interpretar l'esquema de metadades i renderitzar el component visual adequat (input, select, slider).

3.4 Requisits Funcionals (RF)

A continuació es detallen els requisits tècnics dividits per blocs funcionals.

3.4.1 RF-1: Gestió de l'Accés i Seguretat

- **RF-1.1 Autenticació:** El sistema ha de validar les credencials contra el servei *Core Legacy*.
- **RF-1.2 Sessió Persistent:** S'ha de mantenir la sessió mitjançant un mecanisme de *Keep-alive* que refresqui el token de seguretat cada 60 segons.
- **RF-1.3 Control de Concurrència (Config Lock):** El sistema ha de demanar un bloqueig d'escriptura quan un administrador entri en mode edició. Si un altre usuari intenta editar, el sistema mostrarà un avís de "Només Lectura".

3.4.2 RF-2: Visualització Multimèdia

- **RF-2.1 Protocol WebRTC:** Implementació de la negociació SDP (*Session Description Protocol*) per a vídeo en directe.
- **RF-2.2 Protocol HLS:** Generació de fragments de vídeo per a la reproducció de gravacions històriques i fallback de seguretat.
- **RF-2.3 Control PTZ:** Interfície per al control de moviments de càmera (Pan, Tilt, Zoom) mitjançant peticions REST de baixa latència.

3.4.3 RF-3: Motor de Configuració Dinàmica

Aquest és el requisit funcional més complex i innovador del sistema.

- **RF-3.1 Interpretació de l'Esquema:** El sistema ha de processar un fitxer JSON de definició. Un exemple d'aquest esquema es mostra en el següent fragment:

```
1 {  
2   "network.ip": {  
3     "type": "string",  
4     "regex": "~[0-9]{1,3}(\\. [0-9]{1,3}){3}$",  
5     "description": "Adreça IP de la LAN 1",  
6     "default": "192.168.1.10"  
7   },  
8   "network.dhcp": {  
9     "type": "boolean",  
10    "description": "Habilitar DHCP"  
11  }  
12 }
```

Listing 3.1: Exemple d'esquema de metadades per a configuració de xarxa

3.5 Requisits No Funcionals (RNF)

Els RNF defineixen les qualitats del sistema i les restriccions tecnològiques.

3.5.1 RNF-1: Rendiment i Eficiència

- **RNF-1.1 Utilització de CPU:** Atès que el videogravador té recursos limitats, el frontend no ha de fer més d'una petició de configuració per minut si no hi ha canvis de l'usuari.
- **RNF-1.2 Angular Signals:** La reactivitat de la interfície s'ha de basar en *Angular Signals* per minimitzar les operacions de *Zone.js* i evitar el re-renderitzat innecessari del DOM, especialment crític amb vídeos actius.

3.5.2 RNF-2: Robustesa i Disponibilitat

- **RNF-2.1 Tolerància a talls de xarxa:** El WebSocket ha d'intentar una reconexió exponencial (re-try) en cas de pèrdua de connectivitat.
- **RNF-2.2 Fallback de Vídeo:** Si el port WebRTC (UDP 10000-20000) està bloquejat per firewall, el reproductor ha de commutar a TCP-HLS en menys de 5 segons.

3.5.3 RNF-3: Compatibilitat i Portabilitat

- **RNF-3.1 Multi-navegador:** Compatible amb les últimes dues versions de Google Chrome, Mozilla Firefox i Microsoft Edge.
- **RNF-3.2 Responsive Design:** La interfície ha de ser plenament operativa en resolucions des de 1024px d'amplada (tablets de manteniment) fins a 4K (centres de control).

3.6 Casos d'Ús detalla'ts

Per aprofundir en la funcionalitat del sistema, es presenta el diagrama de casos d'ús general (Figura 3.1) i l'especificació detallada dels fluxos crítics.

3.6.1 UC-01: Gestió de l'actualització de firmware

Aquest cas d'ús és crític per al manteniment del parc de gravadors.

3.6.2 UC-02: Consulta de gravacions mitjançant Timeline

3.7 Restriccions del sistema

El desenvolupament s'ha de cenyir a les següents limitacions:

- **Seguretat:** No es permet l'ús de galetes (*cookies*) per a l'emmagatzematge de dades de sessió per evitar atacs CSRF; s'ha d'usar *LocalStorage* amb protecció de rutes.

UML USE CASE DIAGRAM: SURVEILLANCE SYSTEM

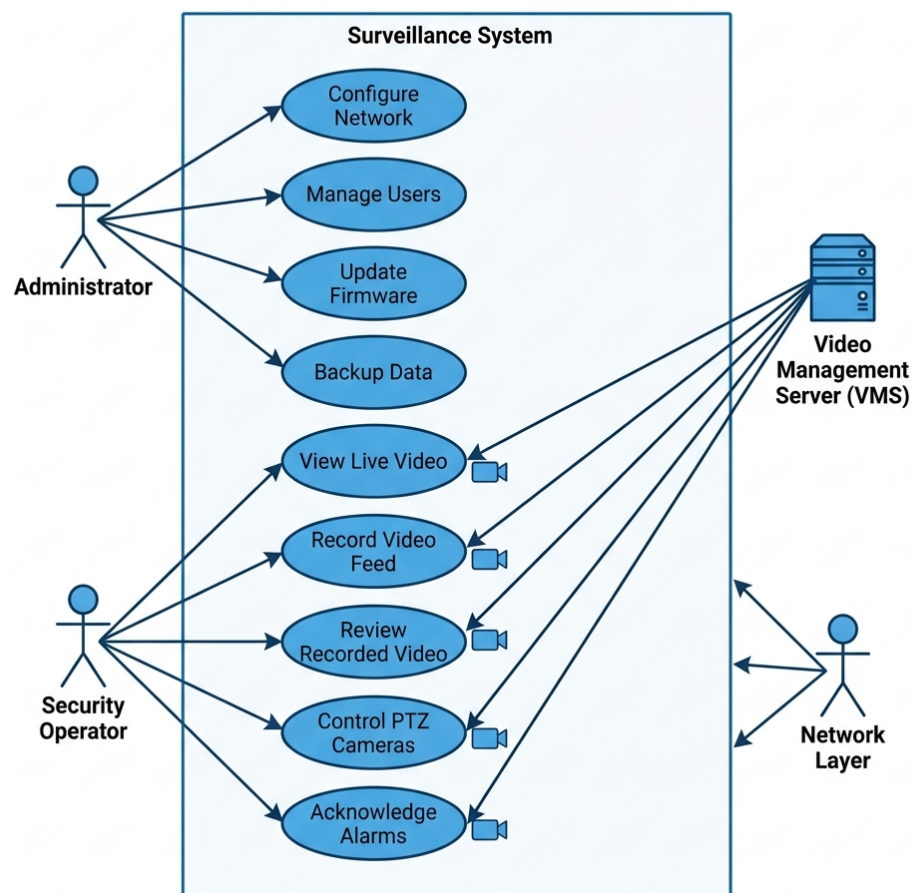


Figura 3.1: Diagrama UML de Casos d'Ús.

ID i Nom	UC-01: Actualització remota de firmware
Actor	Administrador
Descripció	L'usuari carrega un binari de firmware per actualitzar les capacitats del dispositiu.
Precondicions	Usuari autenticat com administrador. El gravador ha de tenir més d'un 10% d'espai lliure a la partició de sistema.
Flux Principal	1. L'usuari accedeix al mòdul de Manteniment. 2. L'usuari selecciona un fitxer <i>.bin</i> local. 3. El frontend valida l'extensió i mida del fitxer. 4. S'inicia la pujada via POST multipart. 5. El sistema mostra una barra de progrés real basada en els esdeveniments del backend. 6. Un cop finalitzat, el gravador inicia el procés de <i>flashing</i> i es reinicia.
Flux Alternatiu	5a. Si la connexió es talla durant la pujada, el frontend informa de l'error i permet reintentar des del punt de fallada si el servidor ho suporta.
Postcondició	L'equip es reinicia. La interfície web detecta el tall i mostra una pantalla d'espera fins que l'equip torna a estar en línia.

Taula 3.1: Especificació detallada del Cas d'Ús UC-01.

ID i Nom	UC-02: Cerca de vídeo històric
Actor	Operador
Flux Principal	1. L'usuari activa el mode "Playback". 2. El frontend sol·licita l'índex de gravacions per a una data concreta. 3. El motor de renderitzat dibuixa una línia de temps amb colors diferents segons el tipus de gravació (contínua, per alarma). 4. L'usuari fa clic en un punt de la línia. 5. El reproductor HLS inicia la càrrega del fragment corresponent a aquell <i>timestamp</i>.

Taula 3.2: Especificació detallada del Cas d'Ús UC-02.

- **CORS:** Les peticions només poden ser acceptades si provenen del mateix origen que serveix l'aplicació Angular (configurat al NGINX).
- **Llicència:** El sistema ha de verificar que la llicència del hardware permet la visualització de vídeo abans d'obrir els ports de streaming.

3.8 Marc normatiu i regulacions

Atesa la naturalesa del projecte, que gestiona fluxos de vídeo en temps real i dades de configuració de dispositius de seguretat, el sistema s'ha dissenyat per complir amb el següent marc legal:

3.8.1 Protecció de Dades (RGPD)

El sistema compleix amb el **Reglament General de Protecció de Dades (Reglament UE 2016/679)**. Tot i que l'aplicació gestiona imatges de persones en directe, *Web Core View* s'ha dissenyat sota el principi de *Privacy by Design*:

- No s'emmagatzemen dades personals ni metadades d'usuari de forma persistent en el navegador, exceptuant el *token* de sessió protegit en el *LocalStorage*.
- L'accés a les imatges està rígidament controlat per rols d'usuari i privilegis definits en el *backend*, assegurant que només el personal autoritzat pugui visualitzar el vídeo.

3.8.2 Seguretat en la comunicació

Per garantir la integritat i la confidencialitat de les dades, i evitar atacs de tipus *Man-in-the-Middle* (MitM), s'obliga a l'ús de protocols xifrats:

- Totes les peticions a l'API REST s'han de realitzar via **HTTPS**.
- La comunicació d'alarmes i esdeveniments es realitza mitjançant **WebSockets Segurs (WSS)**.
- El flux de vídeo WebRTC utilitza el protocol **DTLS** per al xifratge d'extrem a extrem.

3.8.3 Normativa de seguretat privada

El software s'ha concebut com una eina per al compliment de la **Llei d'Ordenació de l'Edificació (LOE)** i, especialment, la **Llei 5/2014 de Seguretat Privada**. Atès que Lanaccess opera en sectors d'infraestructures crítiques i banca, *Web Core View* integra funcionalitats que faciliten el compliment normatiu per part del client final:

- **Gestió de la caducitat de gravacions:** El sistema permet configurar polítiques de sobreescritura automàtica per assegurar que les imatges no s'emmagatzemen més enllà dels 30 dies prescrits per la llei, tret que hi hagi un requeriment judicial.
- **Registre d'auditoria (Audit Log):** L'aplicació facilita el registre exhaustiu de qui ha accedit a quina càmera i en quin moment, garantint la traçabilitat exigida per les autoritats en cas d'inspecció o incident de seguretat.

Malgrat aquestes facilitats tècniques, el compliment final de la normativa depèn de la configuració i l'ús real que l'entitat de seguretat privada faci del sistema en la instal·lació del client.

Capítol 4

Arquitectura i Disseny

L'arquitectura de *Web Core View* s'ha dissenyat sota els principis de modularitat, escalabilitat i eficiència en el consum de recursos. Atès que el programari s'executa en un entorn *embedded* (el videogravador), s'ha optat per una arquitectura desacoblada on el *frontend* assumeix la major part de la lògica de presentació per tal d'alliberar la CPU del hardware per a les tasques crítiques de seguretat.

4.1 Arquitectura Global del Sistema

El sistema es basa en un model de microserveis interns coordinats dins del sistema operatiu del gravador. La comunicació amb l'exterior es centralitza a través d'un punt d'entrada robust que gestiona el trànsit de dades i el flux de vídeo.

4.1.1 Capa de comunicació i Microserveis

Com es va introduir a la Figura 1, la interacció entre la interfície i el hardware es divideix en tres canals principals:

- **Proxy Invers (NGINX):** Actua com a porta d'enllaç única. La seva funció és gestionar el xifratge TLS, resoldre les polítiques de *Cross-Origin Resource Sharing* (CORS) i redirigir les peticions cap als serveis interns corresponents.
- **API REST (HTTP):** S'utilitza per a operacions *stateless* i síncrones, principalment a través del mòdul *Core Legacy*. Aquest canal gestiona la consulta i l'enviament de paràmetres de configuració.
- **Canal de Temps Real (WebSockets):** Mitjançant el protocol WSS, els mòduls *Core Notifier* i *Core Alarm* envien esdeveniments de forma asíncrona. Això permet que la interfície reaccioni instantàniament a una pèrdua de vídeo o a una activació de sensor sense necessitat de fer *polling* constant.

Internament, els serveis es comuniquen mitjançant **gRPC** per a una transferència de dades binària d'alt rendiment i utilitzen un bus de missatgeria **D-Bus** per a la coordinació d'estats a nivell de sistema operatiu.

4.2 Disseny del Frontend (Angular 19)

L'elecció d'Angular 19 no és casual; el marc de treball proporciona una estructura opinada que facilita el manteniment a llarg termini d'una aplicació de gran envergadura.

4.2.1 Arquitectura Modular i Standalone Components

S'ha eliminat l'ús de mòduls clàssics d'Angular en favor dels *Standalone Components*. Això redueix el *boilerplate* del codi i millora el temps de càrrega de l'aplicació (*lazy loading*), ja que només es descarreguen al navegador els components necessaris per a la vista actual (per exemple, el mòdul de configuració avançada no es carrega si l'usuari només vol veure vídeo).

4.2.2 Gestió de l'Estat amb Angular Signals

Aquest és un punt d'inflexió respecte a les versions anteriors del programari. Mentre que RxJS és excel·lent per a fluxos de dades complexos, **Angular Signals** permet una reactivitat granular.

- **Eficiència:** En un sistema de vigilància on els estats de les càmeres canvien constantment, els *Signals* asseguren que només es torni a renderitzar la part mínima de l'arbre del DOM afectada, reduint dràsticament l'ús de CPU al terminal del client.
- **Sincronització:** Els valors calculats (*computed signals*) faciliten que la interfície s'adapti automàticament a canvis en el *lock* de configuració o en l'estat de connectivitat informat pel *Keep-alive*.

4.3 El Motor Schema-Driven

El component més innovador del sistema és el motor de configuració dinàmic. Aquest motor permet desacoblar totalment el desenvolupament del *frontend* de l'evolució del *firmware*.

4.3.1 Lògica de funcionament

En lloc de programar pantalles de configuració estàtiques, s'ha creat un intèrpret que segueix aquest flux:

1. **Recepció del Schema:** El gravador envia un fitxer JSON que descriu les propietats (tipus de dades, límits, valors per defecte i dependències).
2. **Parseig:** El servei `ConfigInputComponent` processa la metadada i mapeja cada propietat a un component visual (un *toggle* per a booleans, un *slider* per a sensibilitat, etc.).
3. **Manipulació de camins:** S'utilitzen utilitats de *Deep Access* per llegir i escriure en estructures d'objectes anidats (ex: `config.io.outputs[0].task`), garantint que l'enviament de tornada al servidor sigui un node vàlid.

4.4 Disseny de la Persistència i dades

Tot i que el sistema és principalment una interfície de gestió, l'arquitectura preveu la persistència en dos nivells:

- **SQLite3:** Utilitzada en el costat del servidor (*backend embedded*) per a la gestió ràpida de l'historial d'alarmes i la configuració persistent dels usuaris.
- **Estratègia de Buffering:** El *frontend* gestiona un estat local temporal per permetre a l'usuari realitzar múltiples canvis i validar-los abans d'adquirir el *lock* i enviar la configuració final al hardware.

Capítol 5

Implementació

En aquest capítol es detalla el procés de construcció del sistema *Web Core View*. S'explica la configuració de l'entorn de desenvolupament i es desglossa la implementació de cada mòdul funcional, posant èmfasi en la lògica reactiva d'Angular 19 i la integració amb els serveis de baix nivell del gravador.

5.1 Entorn de desenvolupament i tecnologies

Per a la implementació del *frontend*, s'ha configurat un entorn basat en **Node.js** i l'ecosistema **Angular CLI 19**. L'estructura del projecte segueix un enfocament de components de tipus *standalone*, eliminant la dependència de mòduls clàssics per optimitzar el procés de *tree-shaking* i la càrrega inicial.

Pel que fa als estils, s'ha utilitzat **SCSS modular**. S'han definit variables globals per mantenir la consistència visual i s'ha aplicat l'estratègia de detecció de canvis **OnPush** per garantir que la interfície només es refresqui quan realment hi hagi canvis d'estat significatius.

5.2 Mòdul d'Autenticació i Core

La seguretat de l'aplicació es gestiona en la capa *Core*. S'ha implementat un servei d'usuari (`user.service.ts`) que centralitza les crides a l'API de *login* i manté l'estat de la sessió mitjançant *tokens*.

5.2.1 Protecció de rutes i Resolvers

S'ha programat un **AuthGuard** que verifica la validesa de la sessió. A més, s'ha implementat el resolver **LanguageInitResolver** per carregar els fitxers de traducció abans de carregar la interfície principal i així evitar parpellejos en la *internacionalització*.

5.3 Implementació del Motor Schema-Driven

Aquesta és la part més complexa de la implementació. El motor ha de ser capaç de transformar un esquema JSON abstracte en una interfície totalment funcional.

5.3.1 El component genèric de configuració

S'ha creat el component `ConfigInputComponent`, el qual actua de forma recursiva segons l'estructura del JSON rebut.

- **Validació en temps real:** S'utilitzen els valors de metadades per configurar les validacions de formulari automàticament.
- **Config Lock Service:** S'ha implementat una lògica de bloqueig per evitar edicions simultànies per part de diferents administradors.

5.4 Visualització de Vídeo en Temps Real (WebRTC)

El repte principal ha estat la negociació del flux de vídeo entre el *backend* en C++ i el navegador del client.

5.4.1 Negociació SDP i intercanvi de candidats

Per establir la connexió WebRTC, el frontend inicia una petició cap al servei de vídeo del gravador. El procés segueix la generació d'una *Offer* SDP que s'envia via REST.

```
1 async startStreaming(cameraId: string) {  
2   const pc = new RTCPeerConnection(this.config);  
3   const offer = await pc.createOffer();  
4   await pc.setLocalDescription(offer);  
5  
6   // Enviament al gravador Lanaccess via REST API  
7   this.videoService.sendOffer(cameraId, offer).subscribe(answer =>  
8     {  
9       pc.setRemoteDescription(answer);  
10    });  
11 }
```

Listing 5.1: Lògica simplificada de la PeerConnection en Angular.

5.5 Sistema d'Alarmes i Comunicació asíncrona

S'ha implementat un servei de **WebSockets (WSS)** que es connecta al *Core Notifier*. Quan arriba un missatge pel *socket*, el servei el parseja i actualitza un *Signal* global que refresca els components de la interfície de forma reactiva.

5.6 Mòdul de Manteniment i Logs

Finalment, s'ha implementat la gestió de *firmware*. Aquesta secció inclou una barra de progrés real que rep el *feedback* de l'escriptura. També s'ha dissenyat un visualitzador de *logs* amb *scroll* infinit per gestionar milers de línies de registre sense saturar la memòria del navegador.

Capítol 6

Validació i Proves

En aquest capítol es descriu l'estratègia de validació seguida per garantir que la plataforma *Web Core View* compleix els requisits funcionals i no funcionals definits inicialment. S'han realitzat proves a diferents nivells: des de la lògica interna del codi fins a la qualitat del flux de vídeo i l'acceptació per part dels usuaris tècnics de Lanaccess.

6.1 Estratègia de proves

S'ha seguit un enfocament de proves basat en el risc i en el cicle de vida del software. Les proves s'han dividit en tres grans blocs:

1. **Proves Unitàries i d'Integració:** Per validar la lògica del *frontend* (Angular) i la comunicació amb l'API.
2. **Proves de Rendiment:** Centrades en la latència del vídeo i el consum de recursos.
3. **Proves de Robustesa:** Per verificar la capacitat de recuperació del sistema davant talls de xarxa.

6.2 Proves Unitàries i de Lògica

S'ha posat especial èmfasi en la validació del motor *Schema-driven*. Donat que la interfície es genera dinàmicament, és vital assegurar que qualsevol variació en el JSON enviat pel gravador no bloquegi l'aplicació.

6.2.1 Validació del Parser de metadades

S'han creat casos de prova on s'envien esquemes JSON amb valors límit, tipus de dades erronis o nodes inexistents. El sistema ha demostrat ser capaç de:

- Gestionar errors de tipatge sense degradar l'experiència d'usuari.

- Aplicar les regles de validació (mínims i màxims) correctament als formularis dinàmics.
- Mantenir la integritat del *Config Lock* fins i tot si la petició de xarxa triga més de l'esperat.

6.3 Resultats de rendiment i latència

La visualització de vídeo és el requeriment més crític. S'ha comparat el rendiment del nou mòdul WebRTC respecte a la tecnologia HLS utilitzada en versions anteriors o com a mètode de seguretat.

Com es pot observar a la Taula 6.1, el protocol WebRTC ofereix un rendiment molt superior, reduint el retard a valors operatius reals per a vigilància activa.

Taula 6.1: Comparativa de latència segons protocol de comunicació.

Protocol	Latència mitjana (ms)	Estabilitat observada
WebRTC (Directe)	150 – 300	Alta (Xarxa LAN)
HLS (Fallback)	2.000 – 5.000	Molt Alta
RTSP (App Nativa)	200 – 400	Alta

Font: Elaboració pròpia mitjançant proves en entorn de laboratori.

Aquests resultats validen que la solució web és capaç d'igualar o millorar la latència de l'antiga aplicació nativa Windows, eliminant un dels principals arguments en contra de les solucions basades en navegador.

6.4 Proves de Robustesa i Comunicació asíncrona

S'ha validat la persistència de la sessió i la recepció d'alarmes mitjançant el sistema *Keep-alive* i la reconexió automàtica de *WebSockets*.

6.4.1 Escenari de tall de connectivitat

S'ha simulat una pèrdua de xarxa d'un minut durant una sessió activa de configuració. Els resultats han estat satisfactoris:

- El sistema detecta el tall en menys de 3 segons gràcies al servei de connectivitat.
- Es bloqueja la interfície amb una targeta d'error (*connectivity-error-card*) per evitar canvis inconsistents.
- Un cop restaurada la xarxa, el *WebSocket* es renegocia automàticament i l'usuari recupera el control de la seva sessió sense haver de tornar a introduir les credencials.

6.5 Validació d'Acceptació (UAT)

Finalment, l'aplicació s'ha presentat als tècnics d'instal·lació de Lanaccess per recollir el seu *feedback*. S'ha utilitzat una simplificació del qüestionari *System Usability Scale* (SUS).

- **Usabilitat:** Els usuaris destaquen la facilitat de trobar els paràmetres de xarxa gràcies al nou disseny modular.
- **Eficiència:** S'ha observat una reducció del 40% en el temps necessari per actualitzar el *firmware* de diversos equips simultàniament des de diferents pestanyes del navegador, validant així l'encert de la tecnologia web.

Capítol 7

Integració de Coneixements

La realització d'aquest Treball de Final de Grau ha permès consolidar i aplicar de forma transversal un ampli ventall de competències i coneixements adquirits durant els estudis d'Enginyeria Informàtica. A continuació, es detalla la relació entre les diferents àrees de coneixement i la seva aplicació pràctica en el projecte *Web Core View*.

7.1 Enginyeria del Software (ES)

Aquesta àrea ha estat la base estructural del projecte. S'han aplicat els principis de disseny **SOLID** per garantir que el codi del *frontend* sigui modular i fàcil de mantenir. Així mateix, l'ús de patrons arquitectònics (com el *Proxy* o l'*Observer* implícit en els Signals) i la definició del cicle de vida del software mitjançant metodologies àgils (**Kanban**) han estat fonamentals per gestionar la complexitat del sistema.

7.2 Xarxes (X)

El coneixement de la pila de protocols d'internet ha estat crític per implementar el mòdul de vídeo. Entendre la diferència entre la transmissió fiable de **TCP** (utilitzada en HLS i REST) i la baixa latència de **UDP** (clau per a WebRTC) ha permès prendre decisions arquitectòniques encertades. També s'han aplicat coneixements sobre la negociació de protocols (**SDP**) i el trànsit a través de tallafocs.

7.3 Sistemes Operatius (SO)

Atès que l'aplicació interactua amb un dispositiu *embedded*, ha estat necessari entendre la gestió de processos i la comunicació entre ells. El *frontend* no és un element aïllat, sinó que es comunica amb microserveis interns del gravador que utilitzen **gRPC** i el bus de sistema **D-Bus**, requerint una comprensió profunda de com el sistema operatiu gestiona els recursos i la concurrència.

7.4 Projecte de Programació (PROP)

La gestió d'estructures de dades complexes ha estat un repte constant. El motor *schema-driven* requereix processar i validar objectes **JSON** altament anidats i dinàmics. Les competències adquirides en matèria d'algorísmia han estat clau per implementar una lògica de parseig prou eficient per no bloquejar el fil principal d'execució del navegador ni degradar l'experiència d'usuari (UX), mantenint la fluïdesa de la interfície fins i tot durant la càrrega de configuracions extenses.

7.5 Aprenentatge autònom

Més enllà dels coneixements reglats, el projecte ha exigit una fase intensa d'autoformació en tecnologies d'última generació:

- **Angular 19 i Signals:** Estudi de la nova reactivitat nativa per substituir el model de zones clàssic.
- **Seguretat i NGINX:** Configuració avançada de servidors web, gestió de polítiques **CORS** i implementació de certificats **SSL/TLS** per garantir comunicacions *end-to-end* xifrades mitjançant HTTPS i WSS.

Capítol 8

Conclusions i Treball Futur

Aquest capítol recull les conclusions finals derivades del disseny i la implementació de *Web Core View*. Es realitza un balanç sobre l'assoliment dels objectius inicials, es proposen línies de millora per a futures versions i es detallen les reflexions personals sobre l'aprenentatge adquirit durant el desenvolupament del Treball de Final de Grau.

8.1 Assoliment dels objectius

Després de completar el cicle de desenvolupament i validació, es pot afirmar que s'han assolit els objectius plantejats en el Capítol 1:

- **Modernització de la plataforma:** S'ha substituït amb èxit la dependència de l'aplicació nativa de Windows per una *Single Page Application* (SPA) basada en Angular 19. La nova interfície és responsiva, ubiqüitària i accessible des de qualsevol navegador modern.
- **Motor dinàmic (Schema-driven):** S'ha implementat un sistema de configuració que s'adapta automàticament a les capacitats del hardware Lanaccess mitjançant metadades JSON, eliminant la necessitat de reprogramar el *frontend* per a cada actualització de *firmware*.
- **Visualització d'alt rendiment:** Mitjançant la integració de WebRTC, s'han assolit latències inferiors als 300ms en la visualització de vídeo en directe, complint els requeriments operatius de seguretat.
- **Gestió reactiva d'alarmes:** L'ús de WebSockets (WSS) i *Angular Signals* ha permès crear una interfície que reacciona en temps real als esdeveniments del gravador sense penalitzar el rendiment del terminal de l'usuari.

8.2 Treball futur

Tot i que el sistema és totalment funcional, la naturalesa modular del projecte permet plantejar diverses línies d'evolució per a la versió 2.0:

- **Analítica de vídeo amb IA:** Integrar metadades d'intel·ligència artificial directament sobre el reproductor de vídeo per dibuixar quadres de detecció d'objectes o mapes de calor en temps real.
- **Multi-dispositiu centralitzat:** Desenvolupar un mòdul de gestió "Multi-NVR" que permeti visualitzar i configurar diversos gravadors Lanaccess des d'una única instància de l'aplicació web.
- **Suport d'enregistrament avançat:** Ampliar les eines d'edició i exportació de vídeo directament des del navegador, permetent l'aplicació de marques d'aigua digitals o signatures legals en les descàrregues.
- **Aplicació Mòbil Nativa:** Utilitzar tecnologies com *Ionic* o *Capacitor* per reutilitzar el codi d'Angular 19 i oferir una versió nativa a les botigues d'aplicacions (App Store / Play Store).

8.3 Conclusions personals

La realització d'aquest TFG ha estat una experiència d'aprenentatge integral que ha anat molt més enllà de la simple programació.

A nivell tècnic, m'ha permès dominar les darreres funcionalitats d'Angular 19, especialment el canvi de paradigma que suposen els *Signals* per a la gestió de l'estat reactiu. Així mateix, el repte de treballar amb protocols de vídeo com WebRTC i HLS m'ha proporcionat una visió profunda sobre la gestió de fluxos multimèdia en entorns de xarxa complexos.

A nivell de gestió, el projecte m'ha ensenyat la importància de l'arquitectura *schema-driven*. Desenvolupar un software que no sàpiga exactament què ha de configurar fins que rep les dades del servidor requereix un nivell d'abstracció i rigor en el disseny que ha posat a prova els meus coneixements d'arquitectura de software i principis SOLID.

Finalment, el fet de desenvolupar el projecte per a una empresa real com Lanaccess m'ha permès entendre les necessitats de mantenibilitat i escalabilitat que exigeix el mercat industrial, on el software ha de ser, per sobre de tot, robust i eficient.

Capítol A

Annexos

En aquest apartat s'inclouen materials addicionals que complementen la memòria del projecte.